Share and import component outputs

For authorised teammates with an existing local `farmx_app` database and PostgreSQL command-line tools. Run these commands yourself in a terminal. The export reads local `.env` credentials; import and access commands prompt for the admin password. The examples use `localhost:5433`; change the host, port or database name if yours differs.

Outputs live in **schemas inside** `farmx_app`, not separate component databases. Exchange only the output schema; keep dumps out of Git and share them privately.

1. Export your own model outputs using `.env`

Run from the repository root with your project's Python environment activated. Install dependencies once:

```
python -m pip install -r backend/requirements.txt
```

In your local `.env`, use your existing `APP_DB_HOST`, `APP_DB_PORT` and `APP_DB_NAME`. **Include the database username and password you have used for your own model outputs** in your component's credential variables. Keep secrets local.

Replace these placeholders in the command below:

• `YOUR_OUTPUT_SCHEMA`: **INCLUDE THE EXACT SCHEMA NAME YOU HAVE USED**, such as `c2_output`.

• `YOUR_ENV_PREFIX`: include the prefix of your component's username/password variables. For example, use `APP_DB_C2` for `APP_DB_C2_USER` and `APP_DB_C2_PASSWORD`, or `APP_DB_PFOI` for `APP_DB_PFOI_USER` and `APP_DB_PFOI_PASSWORD`.

```
python scripts/export_component_outputs.py --schema YOUR_OUTPUT_SCHEMA --env-prefix YOUR_ENV_PREFIX
```

The script loads the root `.env` locally; existing shell variables take precedence. It creates `component_exports/` automatically and saves a timestamped dump there. The terminal prints its exact path. PostgreSQL's `pg_dump` must be installed and available on PATH. The password is passed through the child process environment, not command arguments or printed output. `PFOI_DATABASE_URL` is not used by this export script.

`component_exports/` and `*.dump` are ignored by Git. To verify:

```
git check-ignore component_exports/example.dump
```

Send the successful dump through the team's private channel. **Include the exact output schema name and output table names when sharing the file.** Do not send `core`, input schemas or the whole database. Do not share a dump from a failed export.

2. Import someone else's model outputs

Save the received dump in `component_exports/` and run from the repository root. Replace:

• `THEIR_OUTPUT_SCHEMA`: **INCLUDE THE EXACT SCHEMA NAME THE SENDER HAS USED** for the exported outputs, such as `c3_output`. Put it after `--schema=`. Use the name supplied by the sender, not your own output schema or the dump filename.

• `RECEIVED_FILE.dump`: the exact dump filename you received.

This replaces the local copy of their output schema. Confirm the schema name before running. Your other schemas, including your own inputs and outputs, are outside the selected scope. Only import a trusted teammate's output-schema export from step 1.

```
pg_restore -h localhost -p 5433 -U postgres -d farmx_app --schema=THEIR_OUTPUT_SCHEMA --clean
  --if-exists --no-owner --no-privileges --exit-on-error component_exports/RECEIVED_FILE.dump
```

3. Give your backend and component read access

Replace **every** `THEIR_OUTPUT_SCHEMA` below with the exact schema name supplied by the sender. Replace **every** `YOUR_MODEL_LOGIN` with the database username **you have used for your own component**. Keep `farmx_api` as the backend login, unless your local setup uses a different name.

```
psql -h localhost -p 5433 -U postgres -d farmx_app -c "GRANT USAGE ON SCHEMA THEIR_OUTPUT_SCHEMA TO
  farmx_api, YOUR_MODEL_LOGIN; GRANT SELECT ON ALL TABLES IN SCHEMA THEIR_OUTPUT_SCHEMA TO
  farmx_api, YOUR_MODEL_LOGIN;"
```

The imported objects belong to the restoring admin. Consume the outputs read-only through the producer's documented contract; request a new dump for updates.

4. Check and use the imported outputs

Replace `THEIR_OUTPUT_SCHEMA` with the exact schema name the sender has used, and `THEIR_OUTPUT_TABLE` with an actual output table name supplied by the sender. Keep the dot between them: `schema_name.table_name`. This check returns a count only.

```
psql -h localhost -p 5433 -U postgres -d farmx_app -c "SELECT count(*) AS imported_rows FROM
  THEIR_OUTPUT_SCHEMA.THEIR_OUTPUT_TABLE;"
```

Keep your application's database settings pointing to your existing local database. Follow the producer's integration instructions for any required feature flag or reader configuration; importing tables alone does not enable a UI integration. Check the producer's model/data versions against your local data before using the outputs.

Repeat steps 2–3 whenever you receive an updated dump.